

VR Cinema Web App

Implementation Plan — Personal SBS VR Player

Goal: rebuild the existing GitHub Pages project into a mobile-first VR cinema that can play YouTube URLs, local video files and images, render them in side-by-side (SBS) format, track head movement with the phone gyroscope, and provide independent calibration of the two eye areas for a cardboard VR headset.

Core design principle: treat the left and right eye as two independent environments. Screen zoom/distance changes affect only the cinema screens; the two dark/mask areas remain independently adjustable.

1. Target Architecture

- UI layer** → source selection, playback controls, VR settings, calibration
- State layer** → source type, SBS mode, screen scale, screen distance, eye offsets, masks, gyro settings
- VR renderer** → two eye views, cinema screens, dark environments, camera orientation
- Input adapters** → local video/image, YouTube player, device orientation
- Persistence** → localStorage profile for the physical VR box

2. Recommended Project Structure

```
vr-cinema/ ├── index.html ├── css/ │ └── main.css │ └── controls.css │ └── vr.css └── js/ │ └── app.js │ └── player.js │ └── local-media.js │ └── youtube.js │ └── vr.js │ └── gyro.js │ └── calibration.js │ └── settings.js └── assets/
```

3. Implementation Phases

Phase	Implementation goal
Phase 1 — Existing project cleanup	Separate HTML/CSS/JS responsibilities and remove obsolete UI. Preserve the working GitHub Pages deployment a
Phase 2 — SBS viewport	Split the display into two equal eye regions. Build leftEye and rightEye as separate containers.
Phase 3 — Two dark environments	Give each eye its own fixed dark/mask layer. Do not use one shared black background.
Phase 4 — Cinema screens	Place one independent screen inside each eye environment. Initially use a test image or colored rectangle.
Phase 5 — Local video	Load a selected video with URL.createObjectURL(). Add play, pause, seek and fullscreen.
Phase 6 — 2D and SBS modes	2D mode duplicates the source to both eyes. SBS mode crops/maps the left half to the left eye and right half to the
Phase 7 — Screen-only zoom	Scale the screen objects, not their parent dark areas. Keep the mask/environment geometry unchanged.
Phase 8 — Screen distance	Move the virtual cinema screens along the viewing axis. Keep this separate from screen scale.
Phase 9 — Eye/mask calibration	Add independent left/right mask width, X offset, Y offset and center-gap controls.
Phase 10 — Saved calibration	Persist calibration values in localStorage and add Save, Reset and profile support.
Phase 11 — Gyroscope	Request device orientation permission, convert alpha/beta/gamma to camera orientation, add sensitivity and Recen
Phase 12 — Images	Load JPG/PNG/WebP images and map them to the same screen pipeline.
Phase 13 — YouTube	Integrate the YouTube embed/API as a separate source adapter; do not treat it as a normal local <video> stream.
Phase 14 — Mobile UI	Hide controls while watching, use large touch targets, add an explicit Enter VR / Enable Gyro action.
Phase 15 — Desktop preview	Provide mouse/keyboard look controls so most geometry can be tested without the phone/headset.
Phase 16 — Polish	Add loading states, error handling, settings reset, responsive layout, performance checks and final GitHub Pages de

4. Two-Eye Rendering Model

The application should conceptually maintain this hierarchy:

SBS viewport ■■■■ Left Eye ■ ■■■■ Dark Area / Mask ■ ■■■■ Cinema Screen ■■■■ Right Eye ■■■■ Dark Area / Mask ■■■■ Cinema Screen

This separation is essential. The user wants two dark areas because the physical VR box has two lenses. Screen zoom should change only the screen dimensions. Mask controls should change only the visible dark/eye region.

5. Screen Controls

Control	Purpose
Screen scale	Zoom the cinema screen without scaling the dark area.
Screen distance	Move the virtual screen nearer/farther from the viewer.
Screen X	Fine horizontal alignment.
Screen Y	Fine vertical alignment.
Curvature (optional)	Later feature for a curved cinema/IMAX feel.

6. Independent VR-Box Calibration

Create a calibration screen with a test grid/crosshair. Put the phone in the headset and adjust the image until each lens is centered on its corresponding eye image.

Setting	Why it exists
Left mask width	Controls how much of the left eye area is dark/visible.
Right mask width	Controls how much of the right eye area is dark/visible.
Left X offset	Moves only the left eye content/mask alignment.
Right X offset	Moves only the right eye content/mask alignment.
Y offset	Corrects vertical alignment.
Center gap	Adjusts the separation between the two eye regions.
IPD / eye separation	Optional later control for stereoscopic rendering.

7. Gyroscope / Head Tracking

Use the browser DeviceOrientation API. On supported mobile browsers, request permission from a user-initiated button. Convert the device orientation into camera yaw/pitch/roll. Add a Recenter action so the current phone orientation becomes the neutral forward direction.

DeviceOrientationEvent ↓ permission ↓ alpha / beta / gamma ↓ orientation conversion ↓ camera rotation ↓ left + right eye views

Add gyro sensitivity, optional axis inversion, and a disabled/fallback mode for desktop browsers.

8. Media Pipeline

LOCAL VIDEO File input → Object URL → HTMLVideoElement → VR screen IMAGE File input → Image/Texture → VR screen 2D VIDEO One source → duplicate to left + right eye SBS VIDEO Source → left half / right half → corresponding eye YOUTUBE URL → YouTube embed/API → VR presentation layer

Keep YouTube as a separate adapter because an embedded YouTube player has different browser constraints from a local HTML video element.

9. State Model

```
{ sourceType: "local-video | image | youtube", displayMode: "2d | sbs", screenScale: 1.0, screenDistance: 50, leftOffset: 0, rightOffset: 0, leftMaskWidth: 1.0, rightMaskWidth: 1.0, centerGap: 0, gyroEnabled: false, gyroSensitivity: 1.0 }
```

Store this object in `localStorage` as the user's VR-box profile. This makes calibration persistent across page reloads without needing a backend.

10. Recommended Technology

Layer	Technology
Page/UI	HTML + CSS
Application logic	Vanilla JavaScript
VR/3D scene	Three.js recommended
Local video	HTML <video> + Object URLs
Images	HTML image / Three.js texture
Gyro	DeviceOrientation API
Persistence	localStorage
Hosting	GitHub Pages
Backend	None required for the core local-media player

11. Development Order — Do Not Skip Ahead

1. Get the two eye regions correct.
2. Add the two independent dark areas.
3. Add two test cinema screens.
4. Make local video work.
5. Implement 2D duplication and SBS playback.
6. Implement screen-only zoom.
7. Implement screen distance.
8. Implement independent mask/eye calibration.
9. Save calibration to localStorage.
10. Add gyro and Recenter.
11. Add images.
12. Add YouTube.
13. Build the mobile controls and desktop preview.
14. Polish and deploy.

12. Testing Checklist

- Test portrait/landscape phone orientation.
- Test both eyes with a grid/test image before real media.
- Confirm screen zoom does not enlarge the dark areas.
- Confirm left/right mask controls are independent.
- Confirm Recenter works after turning the phone.
- Test with the phone inside the actual VR box.
- Test the saved calibration after a page reload.
- Test local video sizes and long videos.
- Test an existing SBS movie.

- Test YouTube separately from local video.
- Test fullscreen and browser permission behavior.
- Test on desktop using mouse preview controls.

13. Final UX

The final app should open with a simple source selector. After selecting media, the user enters the SBS cinema view. Controls should be minimal in VR mode: Play/Pause, Screen +/-, Recenter, Gyro, and Settings. Calibration should be a separate screen so normal movie watching stays clean.

End state: YouTube/local video/image → 2D or SBS → two-eye VR cinema → independent dark areas → screen-only zoom → screen distance → lens calibration → saved VR-box profile → gyro head tracking.